

UNIVERSITY OF KIGALI (UoK)

GRADUATE SCHOOL

MACHINE LEARNING

Machine Learning Lecture Notes

Institution	University of Kigali (UoK)
Module Code	MIT91207
Module Title	Machine Learning
Day	1, 2

Academic teaching material prepared for classroom.

Learning Objectives for Day 1 and Day 2

By the end of these chapters, students should be able to:

- Formulate a real-world problem as a machine-learning problem.
- Distinguish classification and regression problems.
- Explain and apply a standard machine-learning workflow.
- Identify and address common data-quality problems.
- Perform exploratory data analysis (EDA).
- Handle missing values appropriately.
- Encode categorical variables and scale numerical variables.
- Identify potential data leakage.
- Perform feature engineering and feature selection.
- Explain correlation and multicollinearity.
- Distinguish statistical association from causation.
- Select appropriate evaluation metrics.
- Explain the principles of Linear Regression, Ridge Regression and Lasso Regression.
- Recognize overfitting and explain methods for improving generalization.
- Use Python, Pandas and Scikit-learn for common ML tasks.
- Interpret model results in the context of a real-world decision.
- Evaluate practical, ethical and operational considerations before deploying an ML system.

Module Overview

Machine Learning (ML) is a branch of Artificial Intelligence that enables computer systems to learn patterns from data and use those patterns to make predictions, classifications, recommendations, or decisions. This module combines conceptual foundations with Python-based practical work. Students should learn not only how to run algorithms, but also how to prepare data, evaluate models, detect common errors, and communicate results responsibly.

Chapter 1 - Foundations of Machine Learning

1.1 Introduction

Learning Objectives

- Define machine learning and explain its main goals.
- Differentiate supervised, unsupervised and reinforcement learning.
- Identify features, labels, training data, validation data and test data.
- Describe a standard end-to-end machine-learning workflow.

1.1.1 What is Machine Learning?

Machine Learning is the study of algorithms that improve their performance on a task through experience represented by data. A conventional program uses explicit rules written by a programmer. An ML system instead estimates useful rules or patterns from examples. For example, rather than writing hundreds of rules for identifying spam, we can train a classifier on emails labelled spam and not-spam.

1.1.2. Core Terminology

An observation is one record or example. A feature is an input variable used by the model. A label or target is the outcome to be predicted. A model is the mathematical function learned from data. Training is the process of estimating model parameters. Inference is the use of the trained model on new data. Generalization describes how well a model performs on previously unseen examples.

1.1.3 Main Types of Learning

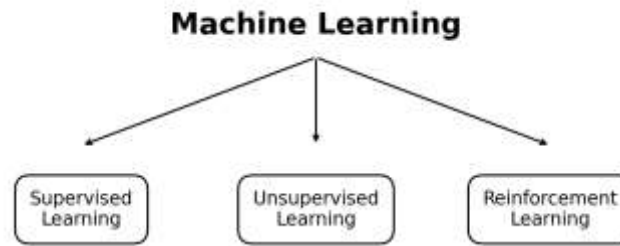


Figure 1. Main categories of machine learning.

Learning Type	Data	Typical Task	Example
Supervised	Labelled	Regression / Classification	Predict student result
Unsupervised	Unlabelled	Clustering / PCA	Segment learners
Reinforcement	Rewards / penalties	Sequential decisions	Agent learns actions

Supervised learning uses labelled examples and includes regression and classification. Regression predicts a numerical value such as house price or monthly sales. Classification predicts a category such as approved/rejected or fraud/not-fraud. Unsupervised learning works with unlabelled data and seeks structure, for example customer segments. Reinforcement learning trains an agent to choose actions through rewards and penalties.

1.1.4 Typical ML workflow



Figure 2. End-to-end machine-learning workflow.

A disciplined workflow includes: define the problem; collect data; understand and clean data; split the dataset; choose features; establish a baseline; train candidate models; evaluate with suitable metrics; tune the model; test once on held-out data; deploy;

monitor performance and data drift. A technically accurate model can still fail if the original problem was poorly defined.

1.1.5 Responsible ML

Machine-learning systems may reproduce bias in historical data, expose private information, or make decisions that are difficult to explain. Students should ask whether the data is lawful and appropriate to use, whether groups are represented fairly, whether human review is required, and whether model performance remains stable after deployment.

1.1.6 Worked / Applied Example

Example: A university wants to predict whether a student may require academic support. Attendance rate, continuous-assessment marks and LMS participation could be features; the support-risk category could be the label. The goal should be early support, not automatic punishment.

1.1.7 Exercises with Answers

1. Define Machine Learning in your own words.

Answer: Machine Learning enables a computer to learn useful patterns from data so that it can make predictions or decisions without every rule being explicitly programmed.

2. Distinguish classification from regression.

Answer: Classification predicts discrete categories; regression predicts continuous numerical values.

3. Give two examples of unsupervised learning.

Answer: Customer/student segmentation using clustering; reducing many correlated variables using dimensionality reduction.

4. Why do we keep a test set separate?

Answer: To obtain a less biased estimate of how the final model performs on unseen data. The test set should not guide routine training decisions.

5. What is generalization?

Answer: The ability of a trained model to perform well on new data that was not used to fit the model.

1.2 Formulating Machine Learning Problems

Learning Objectives

- Define a machine-learning problem from a real-world scenario.
- Distinguish features, targets, observations and predictions.
- Differentiate classification from regression.
- Explain why problem formulation is important.
- Relate model predictions to practical decisions.

1.2.1 From a Real-World Problem to an ML Problem

A machine-learning project normally begins with a practical problem rather than with an algorithm.

For example, an organization may want to:

- predict whether a customer will leave a service;
- estimate the price of a property;
- identify transactions that may be fraudulent;
- predict electricity demand;
- estimate the duration of a journey;
- identify learners who may require additional academic support.

The first task is to translate the practical objective into a **prediction problem**.

A useful formulation identifies:

1. What is being predicted?
2. What information will be available when the prediction is made?
3. What type of value should be predicted?
4. How will the prediction be used?
5. What are the consequences of an incorrect prediction?

The choice of algorithm should come **after** the problem has been properly formulated.

1.2.2 Features and Target

A **feature** is an input variable used by a machine-learning model.

Examples include:

- age;
- income;
- attendance;
- distance;
- temperature;
- number of previous transactions;
- customer rating.

The **target**, also called the label or response variable, is the outcome that the model is required to predict.

For example:

Feature	Feature	Feature	Target
<i>Age</i>	<i>Income</i>	<i>Account duration</i>	<i>Renewal</i>
25	350000	18 months	Yes
41	620000	6 months	No

The features are normally represented by **X**, while the target is represented by **y**.

1.2.3 Classification

Classification predicts a **category or class**.

Examples include:

- approved / rejected;
- fraud / legitimate;
- pass / fail;
- disease / no disease;
- high risk / low risk.

Binary classification has two classes.

Multiclass classification has more than two classes, for example:

- low;
- medium;
- high.

The output of a classifier may be a predicted class and, depending on the algorithm, a probability associated with each class.

1.2.4 Regression

Regression predicts a **numerical value**.

Examples include:

- house price;
- monthly sales;
- electricity consumption;
- temperature;
- journey duration;
- crime rate.

The target is therefore continuous or numerical rather than a category.

1.2.5 Classification or Regression?

The distinction should be based primarily on the **target variable**, not on the subject area.

For example, the same business problem may sometimes be formulated in two ways.

Suppose an organization wants to identify customers who may be at risk. One possible formulation is:

Predict whether a customer will leave during the next three months. This is classification because the target is a category.

Another formulation is: Predict the number of days until a customer leaves. This is regression because the target is numerical.

Therefore, students should examine the target and the intended decision before choosing a modelling approach.

1.2.6 Model Choice Should Follow the Problem

Different algorithms make different assumptions and have different strengths. Possible supervised-learning algorithms include:

Algorithm	Typical Use
Linear Regression	Numerical prediction
Logistic Regression	Classification
Decision Tree	Classification or regression
k-Nearest Neighbours	Classification or regression
Support Vector Machine	Classification or regression
Neural Network	Classification or regression
Ridge Regression	Regression with regularization
Lasso Regression	Regression with regularization

There is rarely one universally best algorithm.

A good model choice considers:

- target type;
- size of the dataset;
- feature types;
- relationships between variables;
- interpretability requirements;
- computational requirements;
- risk associated with errors.

1.2.7 Prediction and Decision Are Not the Same Thing

An ML model produces a prediction. An organization then uses that prediction to support a decision.

For example:

Data → Model → Prediction → Human/organizational decision

A model might predict that a customer has a high probability of leaving. The organization could then decide to offer additional support.

The prediction should not automatically be treated as the final decision.

This distinction is particularly important in areas such as:

- education;
- finance;
- healthcare;
- employment;
- public services.

1.2.8 Factors Beyond Accuracy

Accuracy alone is not always sufficient when evaluating a model. Before deployment, organizations may need to consider:

- interpretability;
- fairness;
- privacy;
- computational cost;
- reliability;
- robustness;
- consequences of false positives and false negatives;
- data quality;
- performance across different population groups;
- ability to monitor the model after deployment.

A highly accurate model may still be unsuitable if its predictions are unfair, unstable or impossible to explain when explanation is required.

1.2.9 Worked / Applied Example

A telecommunications company wants to predict whether customers are likely to cancel their subscriptions.

Possible features include:

- customer age;
- subscription duration;
- monthly expenditure;
- number of previous complaints;
- service usage.

If the target is **cancel / remain**, the problem is classification.

The prediction could be used to identify customers who may benefit from customer-service intervention.

Students should consider not only whether the model predicts correctly, but also whether the prediction can be used responsibly.

1.3 Data Preparation and Exploratory Data Analysis (EDA)

Learning Objectives

- Identify common data-quality problems.
- Explain the importance of data preparation.
- Handle missing and duplicate observations.
- Encode categorical variables.
- Scale numerical variables when appropriate.
- Detect possible data leakage.
- Use EDA to understand a dataset before modelling.
- Select EDA visualizations according to variable types.
- Identify distributions and relationships.
- Compare subgroups.
- Interpret visual patterns.
- Translate EDA findings into evidence for decision-making.

1.3.1 Why Data Preparation Matters

Real-world data is rarely ready for direct use by an ML algorithm. Common problems include:

- missing values;
- duplicate records;
- inconsistent spelling or inconsistent categories;
- incorrect data types;
- extreme observations (outliers);
- irrelevant variables (irrelevant columns);
- different measurement scales;
- categorical variables;
- inconsistent timestamps.

Model quality depends heavily on the quality and representativeness of the data. Data cleaning should therefore be documented and reproducible. Data preparation attempts to make the dataset suitable for analysis and modelling.

A typical process is:

Raw data → Data inspection → Cleaning → Transformation → Feature engineering → Model-ready data

1.3.2 Inspecting a Dataset

Initial inspection should establish:

- number of observations;
- number of variables;
- variable names;
- data types;
- missing values;
- duplicate records;
- numerical distributions;
- categorical values;
- possible outliers.

Useful Pandas commands include:

```
import pandas as pd

df = pd.read_csv("data.csv")

print(df.head())
print(df.shape)
print(df.info())
print(df.describe())
print(df.isna().sum())
print(df.duplicated().sum())
For categorical variables:
print(df["category"].value_counts())
```

1.3.3 Missing Values/ Missing data

Missing values can occur for many reasons. For example:

- a measurement was not taken;
- a respondent did not provide information;
- a system failed to record a value;
- a variable was not applicable;
- information was intentionally withheld.

Therefore, missingness should not automatically be treated as zero. Common strategies for dealing with missing data are:

a. Remove observations/records

Appropriate when:

- only a small number of records are affected (missingness is rare in the dataset);
- missingness is non-systematic or does not introduce important bias;
- enough observations remain.

b. Mean imputation

A missing numerical value is replaced by the mean. This is simple but can be affected by outliers.

c. Median imputation

The missing value is replaced by the median. This is often more robust when the distribution is skewed or contains outliers.

d. Mode imputation

A missing categorical value can sometimes be replaced by the most frequent category.

e. 'Unknown' category

For some categorical variables, missingness itself may contain information. An 'Unknown' category can therefore be appropriate.

The appropriate strategy depends on:

- variable type;
- amount of missingness;

- distribution;
- reason for missingness;
- consequences of an incorrect replacement.

1.3.4 Why Replacing Missing Values with Zero Can Be Dangerous

Zero is a real numerical value. Suppose the variable is Annual income.

- A value of zero could mean: The person's income is actually zero.
- A missing value means: The income is unknown or was not recorded.

These are not necessarily equivalent.

The same problem occurs with:

- age;
- number of previous transactions;
- loan amount;
- medical measurements;
- examination marks;
- distance travelled.

Students should therefore ask: Does zero have a meaningful interpretation for this variable? If not, zero may be an inappropriate imputation value.

1.3.5 Duplicate Records

Duplicate records may occur because:

- data was imported twice;
- a transaction was recorded repeatedly;
- multiple systems were merged;
- records were appended incorrectly.

Duplicates can distort:

- distributions;
- class frequencies;

- correlations;
- model training.

Pandas provides:

```
df.duplicated().sum()
```

and:

```
df = df.drop_duplicates()
```

However, duplicate removal should be based on an understanding of what constitutes a genuine duplicate.

1.3.6 Inconsistent Categories

Consider a variable called region containing:

Kigali

kigali

KIGALI

Kigali

These values may represent the same category.

A simple cleaning operation may be:

```
df["region"] = (
    df["region"]
    .str.strip()
    .str.lower()
)
```

Category cleaning should be performed carefully because different labels may sometimes have genuinely different meanings.

1.3.7 Categorical Variables

Machine-learning algorithms often require numerical input. Categorical variables can be divided into:

- **Nominal variables**

Categories have no natural order.

Examples:

- region;
- occupation;
- payment method;
- road type.
- **Ordinal variables**

Categories have a meaningful order. For examples:

- low / medium / high;
- poor / fair / good / excellent.

The encoding method should reflect this distinction.

1.3.8 Encoding Categorical Variables

Most ML algorithms require numerical inputs. **Ordinal encoding** is suitable when categories have a genuine order, while **one-hot encoding** creates indicator columns and is safer for unordered categories such as district or programme. **Target encoding** replaces categories with target-based statistics and can be useful for high-cardinality categorical variables; however, use **smoothed target encoding** to reduce the effect of rare categories. Assigning arbitrary numbers to nominal categories can incorrectly suggest an order.

1.3.9 One-Hot Encoding

One-hot encoding creates a binary variable for each category.

For example:

Road type
Local
Highway
Arterial

can become:

Road_Local	Road_Highway	Road_Arterial
1	0	0
0	1	0
0	0	1

This avoids creating an artificial numerical ordering.

In Scikit-learn:

```
from sklearn.preprocessing import OneHotEncoder
encoder = OneHotEncoder(handle_unknown="ignore")
```

1.3.10 Numerical Encoding of Categories

Suppose:

Local = 0

Arterial = 1

Highway = 2

For a nominal variable, this can incorrectly suggest:

Highway > Arterial > Local

The numerical values may be interpreted as meaningful distances even though the categories have no such relationship. Therefore, arbitrary numerical coding should generally not be used for nominal categories.

1.3.11 Feature Scaling

Variables/Features can have very different numerical ranges.

For example:

- age: 18–80;
- income: 100,000–5,000,000;
- distance: 1–30.

Some algorithms are sensitive to these differences. Different feature scaling exist, such as standardization and min-max scaling.

- **Standardization**

Standardization transforms a variable to approximately zero mean and unit standard deviation. Standardization transforms a feature approximately according to:

$$z = \frac{x - \mu}{\sigma}$$

where:

- x is the original value;
- μ is the training mean;

- σ is the training standard deviation.

In Python:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
```

- **Min-Max Scaling**

Min-max scaling maps values to a fixed interval, commonly 0 to 1 (interval $[0,1]$):

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Scaling is particularly important for:

- k-nearest neighbours;
- support vector machines;
- regularized linear models;
- neural networks;
- many gradient-based methods.

Decision trees generally do not require feature scaling.

1.3.12 Train-Test Split

The dataset is commonly divided into:

- training set;
- validation set;
- test set.

The **training set** is used to fit the model.

The **validation set** can be used to compare models and tune hyperparameters.

The **test set** provides a final estimate of performance on unseen data.

A common split might be:

- 70% training;

- 15% validation;
- 15% testing.

Alternatively, a two-way split into **training test** and **test set** may be used, commonly with 80% of the data for training and 20% for testing, particularly when cross-validation (CV) is applied to the training set for model selection and tuning. CV is especially useful when the dataset is small or moderate in size, as it provides a more reliable estimate of model performance. 80/20 is a common rule of thumb, not a strict requirement. Depending on the dataset size and problem, 70/30, 75/25, or 90/10 may also be appropriate.

1.3.13 Data Leakage

Data leakage occurs when information that should not be available to the model enters the model-development process.

- For example, imagine predicting whether a customer will cancel a subscription next month.

A variable recorded **after** cancellation should not be used to make the prediction.

Another example is using a final graduation status to predict first-year dropout risk would leak future information.

- Leakage can also occur during preprocessing.

Suppose the entire dataset is standardized before the train-test split. Then the test data has influenced the calculation of the mean and standard deviation. Preprocessing parameters such as means, medians and scaling factors should be learned from the training set and then applied to validation/test data.

The correct approach is:

Split → Fit preprocessing on training data → Transform training and test data

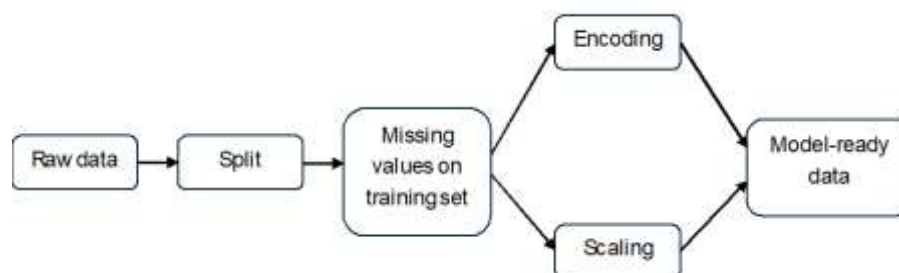


Figure 3. Data-preparation pipeline that prevents data leakage.

1.3.14 Exploratory Data Analysis

EDA is the systematic examination of data before or during modelling.

EDA is an investigative process. EDA should not begin with: Which graph should I draw?

It should begin with: What do I need to understand about the data?

Examples:

- Is the target evenly distributed?
- Are some categories much more common?
- Does the target differ between groups?
- Are two numerical variables related?
- Are there extreme observations?
- Does the relationship appear linear?

Important EDA activities include:

- summary statistics;
- distributions;
- category frequencies;
- missing-value analysis;
- outlier detection;
- pairwise relationships;
- correlation analysis;
- subgroup comparisons;
- visual inspection.

EDA Technique	Purpose	Typical Output
Summary statistics	Understand center and spread	Mean, median, standard deviation
Histogram	Inspect distributions	Skewness and unusual values
Correlation matrix	Explore numeric associations	Potential relationships
Category counts	Check balance	Class/programme frequencies

EDA helps detect skewness, imbalance, unusual values and potentially useful relationships. Correlation does not establish causation, and a high correlation among input variables may create multicollinearity problems in some models.

EDA should answer questions such as:

- What does the dataset contain?
- Are there quality problems?
- Which variables are numerical or categorical?
- Are variables strongly skewed?
- Are some categories rare?
- Are there unusual observations?
- Which variables appear related?

EDA is not simply a collection of graphs. Each visualization should have a purpose.

1.3.15 Common EDA Visualizations

Visualization	Suitable Purpose
Histogram	Numerical distribution
Boxplot	Spread and possible outliers
Bar chart	Category frequencies
Scatterplot	Relationship between two numerical variables
Grouped bar chart	Compare categories across groups
Correlation heatmap	Numerical associations
Line plot	Trends over time

a) Numerical Variable Distribution

A histogram can show the distribution.

```
plt.hist(df["income"], bins=20)
plt.xlabel("Income")
plt.ylabel("Frequency")
plt.show()
```

Look for:

- skewness;

- concentration;
- multiple peaks;
- extreme values.

A boxplot to show its distribution, median, quartiles, spread, and potential outliers.

```
plt.boxplot(df['price'])
```

A boxplot to compare distributions of several numerical variables and identify differences in scale or outliers.

```
df[['price', 'income', 'age']].boxplot()
```

b) Categorical Variable

A bar chart can show category frequencies.

```
df["occupation"].value_counts().plot(kind="bar")
plt.xlabel("Occupation")
plt.ylabel("Frequency")
plt.show()
```

This can reveal:

- dominant categories;
- rare categories;
- possible imbalance.

c) Numerical Variable versus Numerical Variable (relationship)

Use a scatterplot.

```
import matplotlib.pyplot as plt

plt.scatter(df["advertising_spend"], df["sales"])
plt.xlabel("Advertising spend")
plt.ylabel("Sales")
plt.show()
```

Look for:

- positive or negative association;
- nonlinear patterns;
- clusters;
- outliers;

changing variability.

d) Numerical Variable Across Categories/groups (relationship)

A boxplot can compare a numerical variable across groups to compare distributions between groups. For example:

```
df.boxplot(
    column="income",
    by="occupation"
)
plt.show()
```

This can reveal differences in:

- median;
- spread;
- outliers.

Choosing Visualizations:

Variable relationship	Useful visualization
Numerical → Numerical	Scatterplot
Numerical distribution	Histogram
Numerical → Groups	Boxplot
Categorical frequency	Bar chart
Numerical relationships	Correlation heatmap
Time → Numerical	Line plot

The visualization should answer a specific analytical question.

1.3.16 Interpretation

A weak interpretation is: The graph shows sales and advertising.

A stronger interpretation identifies the observed pattern: Sales generally increase as advertising expenditure increases, although the relationship becomes less consistent at higher expenditure levels.

The strongest interpretations also consider:

- possible explanations;
- limitations;
- outliers;
- whether the relationship implies causation.

1.3.17 Worked / Applied Examples

1. A retailer has customer information containing:

- age;
- income;
- occupation;
- region;
- purchase frequency;
- satisfaction.

Before modelling, the analyst should determine:

- whether values are missing;
- whether duplicate customers exist;
- whether occupation labels are consistent;
- which variables are categorical;
- whether numerical variables require scaling;
- whether the target is available at the time of prediction.

Only after this investigation should the final preprocessing pipeline be constructed.

2. In a loan dataset, income may contain missing values, *employment_type* may be categorical, and *annual_income* may range from thousands to millions. A preprocessing pipeline can impute, encode and scale consistently without leaking test information.

1.3.18 Exercises with Answers

1. What is data leakage?

Answer: Leakage is the use of information during model development that would not legitimately be available when the model makes a real prediction.

2. When is median imputation often preferred to mean imputation?

Answer: When the numerical variable is skewed or contains strong outliers.

3. Why is one-hot encoding useful?

Answer: It converts nominal categories into numerical indicators without introducing a false numeric order.

4. Name two algorithms for which scaling is usually important.

Answer: Examples include k-nearest neighbors, support vector machines, logistic/linear models with regularization, and neural networks.

5. Does correlation prove causation?

Answer: No. Correlation measures association, not a causal relationship.

1.4 Feature Engineering and Feature Selection

Learning Objectives

- Explain feature engineering.
- Construct meaningful features from existing variables.
- Distinguish useful, redundant and irrelevant variables.
- Explain correlation and multicollinearity.
- Understand why feature selection should be based on more than correlation alone.

1.4.1 What Is Feature Engineering?

Feature engineering involves creating, transforming or combining variables to provide useful information to a model.

For example, suppose a dataset contains:

- date of birth;
- current date.

Instead of using both directly, an analyst might derive: Age

Another example is:

- total purchases;
- number of months as a customer.

These can be used to construct: Average purchases per month

A useful, engineered feature should have a meaningful relationship with the prediction task.

1.4.2 Behavioural Features

Behavioural data can often be transformed into more informative measures. For example, from online-learning activity one might construct:

- number of courses completed;
- completion rate;
- average rating;

- number of active learning days;
- time since last activity;
- average courses per month.

The important question is not simply: Can I create another variable?

It is: Does this variable provide useful information that is available at prediction time?

1.4.3 Irrelevant Variables

Some variables may have little or no meaningful relationship with the target.

Examples include:

- record identifiers;
- randomly generated values;
- administrative fields;
- variables unrelated to the prediction task.

An identifier such as:

1001

1002

1003

1004

should not normally be treated as a meaningful predictor merely because it is numerical.

1.4.4 Feature Selection

Feature selection is the process of deciding which available variables should be retained for modelling.

Possible considerations include:

- domain knowledge;
- predictive usefulness;
- missingness;
- redundancy;

- multicollinearity;
- interpretability;
- computational cost;
- data availability at prediction time.

Feature selection should not be based on one criterion alone.

1.4.5 Correlation

Correlation measures the strength and direction of association between two numerical variables.

Pearson correlation is commonly represented by:

$$r = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y}$$

The value ranges approximately from:

$$-1 \leq r \leq 1$$

Interpretation:

- r close to +1: strong positive linear association;
- r close to -1: strong negative linear association;
- r close to 0: weak linear association.

1.4.6 Correlation Does Not Establish Causation

Suppose two variables are strongly correlated. This does not necessarily mean:

X causes Y.

Possible explanations include:

- Y causes X;
- another variable causes both;
- the relationship is coincidental;
- the relationship exists only within the observed dataset.

Machine-learning prediction and causal inference are therefore different tasks.

1.4.7 Correlation Matrix

A correlation matrix summarizes pairwise correlations among numerical variables.

Example:

```
corr = df.select_dtypes(include="number").corr()
print(corr)

# A heatmap can make the matrix easier to inspect:
import matplotlib.pyplot as plt

plt.imshow(corr, cmap="coolwarm")
plt.colorbar()
plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
plt.yticks(range(len(corr.columns)), corr.columns)
plt.tight_layout()
plt.show()
```

Students should not merely produce the matrix. They should explain what important patterns mean.

1.4.8 Multicollinearity

Multicollinearity occurs when predictor variables are strongly related to each other.

For example:

$$X_1 \approx X_2$$

If two predictors contain very similar information, it can become difficult to determine the individual contribution of each predictor.

Multicollinearity can cause:

- unstable regression coefficients;
- large coefficient standard errors;
- difficulty interpreting coefficients;
- sensitivity to small changes in the data.

It does not necessarily mean that prediction will always be poor.

1.4.9 Possible Responses to Multicollinearity

Depending on the problem, an analyst may:

- remove one of the redundant variables;
- combine variables;
- use dimensionality reduction;
- use regularization;
- retain the variables if prediction is the primary objective and the model remains satisfactory.

The appropriate strategy depends on the purpose of the model.

1.4.10 Feature Selection Is Not Just Correlation

A variable with low correlation with the target may still be useful.

For example:

- its relationship may be nonlinear;
- it may interact with another feature;
- it may be useful only within a subgroup.

Similarly, a variable with high correlation may be redundant because another predictor contains almost the same information.

Therefore: Correlation is a useful diagnostic, not a complete feature-selection method.

1.5 Regression and Model Evaluation

Learning Objectives

- Explain simple and multiple linear regression.
- Interpret regression predictions and residuals.
- Distinguish training and test performance.
- Calculate and interpret common regression metrics (MAE, MSE, RMSE, and R-squared).
- Recognize overfitting, underfitting and non-linear relationships.

1.5.1 Regression problem

Regression predicts a continuous target. Examples include revenue, temperature, electricity demand and house price.

1.5.2 Regression

Regression predicts a numerical target. A simple linear regression model can be written as:

$$\hat{y} = \beta_0 + \beta_1 x$$

The multiple regression extends the equation to more than one feature:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

where:

- $\hat{y}$ is the predicted value;
- β_0 is the intercept;
- β_i are model coefficients;
- x_i are predictor variables.

1.5.2 Ordinary Least Squares

Ordinary Least Squares (OLS) estimates coefficients by minimizing the sum of squared residuals.

A residual is:

$$e_i = y_i - \hat{y}_i$$

OLS minimizes:

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Large errors receive greater influence because they are squared. A residual is the observed value minus the predicted value. The coefficient sign indicates the direction of association while its magnitude depends on units and the presence of other variables.

The process of estimating the coefficients of a linear model is called fitting the linear model. Regression analysis involves both fitting the model and interpreting its coefficients.

1.5.3 Regression Metrics

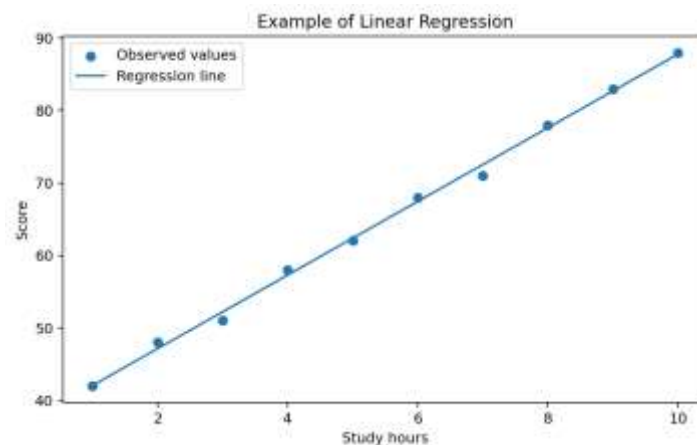


Figure 4. Illustrative linear-regression relationship.

- **Mean Absolute Error**

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

MAE represents the average absolute prediction error in the target's units. If the target is measured in minutes and MAE is 4, predictions are off by approximately 4 minutes on average in absolute terms.

- **Mean Squared Error**

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

MSE penalizes large errors more strongly.

- **Root Mean Squared Error**

$$RMSE = \sqrt{MSE}$$

RMSE is expressed in the same units as the target.

- **R-squared**

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

R-squared indicates how much variation is explained relative to a mean-prediction baseline. R-squared can be negative on test data when a model performs very poorly. A high R-squared does not automatically mean that a model is appropriate for deployment.

Metric	Meaning	Interpretation
MAE	Average absolute error	Easy to explain in target units
MSE	Average squared error	Penalizes large errors
RMSE	Square root of MSE	Error in target units
R-squared	Variance explained vs baseline	Higher is generally better

1.5.4 Choosing a Regression Metric

Different metrics answer different questions.

Metric	Main characteristic
MAE	Easy interpretation of typical absolute error
MSE	Strong penalty for large errors
RMSE	Error in target units with greater sensitivity to large errors
R ²	Relative explanatory/predictive performance

A good evaluation should consider the practical consequences of errors rather than selecting a model using one number without context.

1.5.5 Training and Test Performance

Suppose a model achieves:

- very low training error;
- much higher test error.

This may indicate **overfitting**.

Overfitting occurs when a model learns patterns that are too specific to the training data and therefore does not generalize well. **Underfitting** occurs when the model is too simple to capture important patterns. The desired situation is good performance on unseen data.

1.5.6 Baseline Models

- A baseline provides a simple reference point.
- For regression, a basic baseline might predict the training-set mean for every test observation.
- A sophisticated model should provide meaningful improvement over a simple baseline.
- Without a baseline, it can be difficult to determine whether a model is genuinely useful.

1.5.7 Assumptions and Diagnostics

Classical linear-regression interpretation commonly considers linearity, independent errors, constant error variance and limited multicollinearity. For predictive work, residual plots are useful for detecting systematic patterns. A curved residual pattern suggests that a simple linear form may be inadequate.

1.6.9 Human Oversight

In high-impact applications, an ML prediction should not necessarily become an automatic decision. A system may instead:

Model prediction → Human review → Final decision

This can be particularly important where incorrect decisions may significantly affect an individual's:

- education;
- health;
- financial access;
- employment;
- legal or public-service outcome.

1.6 Regularization: Ridge and Lasso Regression

Learning Objectives

- Explain why regularization is used.
- Compare OLS, Ridge and Lasso.
- Explain L1 and L2 penalties.
- Understand the effect of the regularization parameter.
- Recognize the importance of feature scaling.

1.6.1 Why Regularization?

A model with many predictors may become unstable or overfit the training data. Regularization modifies the optimization objective by adding a penalty for large coefficients. This encourages simpler models.

Two common regularized linear models are:

- Ridge Regression;
- Lasso Regression.

1.6.2 Ridge Regression

Ridge uses an L2 penalty.

Ridge regression adds an L2 penalty that shrinks coefficients.

Conceptually, its objective is:

$$\text{SSE} + \alpha \sum_{j=1}^p \beta_j^2$$

where:

- SSE is the sum of squared errors;
- β_j are model coefficients;
- α controls the strength of regularization.

Increasing α generally causes stronger coefficient shrinkage.

Ridge is particularly useful when predictors are correlated. It can stabilize models with correlated features. The penalty strength α is a hyperparameter and should be selected using validation or cross-validation.

1.6.3 Lasso Regression

Lasso uses an L1 penalty:

$$\text{SSE} + \alpha \sum_{j=1}^p |\beta_j|$$

One important characteristic of Lasso is that some coefficients may become exactly zero. This can provide a form of feature selection. Same as ridge regression, the penalty strength is a hyperparameter and should be selected using validation or cross-validation.

1.6.4 Ridge versus Lasso

Property	Ridge	Lasso
Penalty	L2	L1
Shrinks coefficients	Yes	Yes
Can produce exact zero coefficients	Usually no	Yes
Useful with correlated predictors	Often	Can select one or a subset
Provides feature selection	Not directly	Yes

Neither method is automatically better. The appropriate model depends on:

- data structure;
- predictive performance;
- feature relationships;
- desired interpretability;
- regularization strength.

1.6.5 The Alpha Parameter

In Scikit-learn:

```
from sklearn.linear_model import Ridge, Lasso

ridge = Ridge(alpha=1.0)
lasso = Lasso(alpha=1.0)
```

The value of alpha controls regularization strength. A very small value produces weak regularization. A very large value can shrink coefficients excessively and lead to underfitting. Therefore, alpha should normally be selected using validation or cross-validation rather than arbitrarily.

1.6.6 Why Feature Scaling Matters for Ridge and Lasso

Given that such regularizations penalize coefficients, if two features have very different scales, their coefficients are not directly comparable.

For example:

- Feature A ranges from 0 to 1;
- Feature B ranges from 0 to 1,000,000.

Standardization is therefore commonly used before Ridge or Lasso.

A pipeline can combine preprocessing and modelling:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge

ridge_model = Pipeline([
    ("scaler", StandardScaler()),
    ("model", Ridge(alpha=1.0))
])
```

1.6.7 Cross-Validation for Ridge and Lasso

Cross-validation helps estimate how well any model is likely to generalize and can be used to select hyperparameters such as the alpha parameter.

In k-fold cross-validation:

1. Divide the training data into k folds.
2. Train on k-1 folds.
3. Validate on the remaining fold.
4. Repeat until every fold has been used for validation.
5. Average the results.

For example:

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(
    ridge_model,
    X_train,
    y_train,
    cv=5,
    scoring="neg_mean_absolute_error"
)
print(scores)
```

The test set should remain untouched until the final evaluation.

1.6.8 Worked / Applied Examples

1. Suppose a company wants to predict delivery time using:

- distance;
- traffic;
- weather;
- road characteristics;
- previous delivery time.

Two variables may contain highly similar traffic information. OLS may produce unstable coefficients. Ridge can reduce coefficient instability through L2 regularization. Lasso may shrink some coefficients to zero.

The final choice should be based on validation evidence and the practical requirements of the application.

2. Predict monthly electricity consumption from household size, number of appliances and average daily operating hours. A regression model produces a numerical kWh estimate, and MAE tells us the typical absolute prediction error.

1.6.9 Exercises with Answers

1. What type of target is suitable for regression?

Answer: A continuous numerical target such as price, demand, age, score or temperature.

2. If $MAE = 8.5$ for a model predicting marks, how can it be interpreted?

Answer: Predictions differ from actual marks by about 8.5 marks on average in absolute terms.

3. Why does MSE emphasize large errors?

Answer: Because each error is squared before averaging.

4. What does a negative test R-squared indicate?

Answer: The model is performing worse than a simple baseline that predicts the test target mean.

5. Compare Ridge and Lasso briefly.

Answer: Both regularize coefficients; Ridge uses L2 shrinkage, while Lasso uses L1 and can set some coefficients exactly to zero.

1.7 Time, Data Leakage and Reliable Model Evaluation

Learning Objectives

- Explain temporal data leakage.
- Distinguish random splitting from time-aware splitting.
- Understand why future information must not enter training.
- Design a reliable evaluation procedure.

1.7.1 Why Time Matters

Many datasets contain timestamps. Examples include:

- transactions;
- online activity;
- patient records;
- financial applications;
- journeys;
- sensor measurements.

When making a prediction about the future, information from the future must not be used to train the model in a way that would not be available in practice.

1.7.2 Temporal Leakage

Suppose the task is: Predict next month's customer behaviour.

A variable calculated using information from next month would be inappropriate. Similarly, using an outcome that occurred after the prediction date can introduce leakage.

1.7.3 Time-Aware Splitting

For some problems, a chronological split is more appropriate:

Earlier observations → Training

Later observations → Validation/Test

For example:

2019–2023 → Training

2024 → Validation

2025 → Test

The exact periods depend on the application.

1.7.4 Why Random Splitting Can Be Misleading

Random splitting assumes that observations can reasonably be treated as independent samples from the same distribution. With time-dependent data, nearby observations may be strongly related. Randomly mixing past and future observations can therefore produce an unrealistically optimistic estimate of performance.

1.7.5 Practical Question

Before using a variable, ask: Would this information genuinely be available at the exact moment when the prediction is made?

If the answer is no, the variable should not be used.

1.8 Building Reproducible Machine-Learning Pipelines

Learning Objectives

- Explain why reproducible pipelines are important.
- Combine preprocessing and modelling.
- Apply identical transformations to training and new data.
- Avoid inconsistent deployment transformations.

1.8.1 What Is a Pipeline?

A machine-learning pipeline connects data transformations and modelling steps. For example:

Raw data → Imputation → Encoding → Scaling → Model

This ensures that the same transformations can be applied consistently.

1.8.2 Example Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge

pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("model", Ridge(alpha=1.0))
])

pipeline.fit(X_train, y_train)
predictions = pipeline.predict(X_test)
```

The pipeline learns the preprocessing parameters from the training data.

1.8.3 Mixed Numerical and Categorical Data

When a dataset contains both numerical and categorical variables, a `ColumnTransformer` can be used.

```

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("numeric", numeric_pipeline, numerical_columns),
    ("categorical", categorical_pipeline, categorical_columns)
])

```

1.8.4 Consistency During Deployment

Suppose a model was trained using:

- median imputation;
- one-hot encoding;
- standardization.

A new record arriving after deployment must go through the **same transformations**.

The model should not suddenly receive raw or differently encoded variables. This is one reason pipelines are valuable.

1.8.5 Unknown Categories

New data may contain a category that was not present during training. For example, a new region may appear after the model is deployed.

Using:

```
OneHotEncoder(handle_unknown="ignore")
```

can prevent the pipeline from failing simply because an unseen category appears.

1.9 From ML Results to Real-World Recommendations

Learning Objectives

- Interpret ML results in context.
- Distinguish prediction from causal explanation.
- Consider model limitations.
- Translate analytical findings into practical recommendations.

1.9.1 Statistical Result versus Practical Meaning

Suppose a model indicates that variable x is strongly associated with target y .

The analyst should ask:

- How strong is the relationship?
- Is it consistent?
- Is it practically important?
- Could another variable explain the relationship?
- Does the relationship support prediction?
- Is there evidence that changing X would change Y ?

These questions prevent overinterpretation.

1.9.2 Association versus Causation

An observational dataset can identify associations. For example:

Areas with higher x tend to have higher y .

This does not necessarily establish: Increasing x causes y to increase.

Causal conclusions normally require stronger research designs or additional evidence.

1.9.3 Recommendations Should Be Evidence-Based

A recommendation should connect:

Evidence → Interpretation → Action

For example:

1. Analysis identifies a consistent pattern.
2. The pattern is interpreted cautiously.
3. The organization considers possible actions.
4. Additional evidence is considered before implementing a major intervention.

1.9.4 Limitations

Every ML study has limitations. Examples include:

- limited sample size;
- biased sampling;
- missing variables;
- measurement errors;
- historical data that may no longer represent current conditions;
- synthetic rather than real-world data;
- correlation without causal evidence;
- model instability;
- poor subgroup representation.

A strong analytical report acknowledges relevant limitations instead of presenting predictions as certainty.

1.10 Practical Python Workflow for Machine Learning

Learning Objectives

- Load and inspect a dataset.
- Separate features and target.
- Split data.
- Train a model.
- Generate predictions.
- Calculate evaluation metrics.
- Compare models.

1.10.1 Basic Workflow

A general Scikit-learn workflow is:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

df = pd.read_csv("data.csv")

X = df.drop(columns=["target"])
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
mae = mean_absolute_error(y_test, predictions)
print("MAE:", mae)
```

The exact preprocessing and model depend on the dataset.

1.10.2 Comparing Models

When comparing models, create a consistent evaluation procedure.

For example:

```
models = {
    "Linear Regression": LinearRegression(),
    "Ridge": Ridge(alpha=1.0),
    "Lasso": Lasso(alpha=1.0)
}
for name, model in models.items():
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    mae = mean_absolute_error(y_test, predictions)
    print(name, mae)
```

However, direct comparison requires that preprocessing is appropriate for all models. For regularized models, scaling should normally be included in the pipeline.

1.10.3 Model Comparison Should Go Beyond One Number

Suppose:

Model	MAE	RMSE	R^2
Model A	4.8	7.2	0.82
Model B	5.0	6.5	0.85
Model C	5.1	6.8	0.84

There is no automatic rule that says: Choose the model with the lowest MAE.

The analyst should consider:

- which errors matter most;
- whether large errors are especially harmful;
- generalization;
- model complexity;
- interpretability;
- stability;
- computational requirements.

The final choice should be justified.

1.11 A Complete Machine-Learning Problem-Solving Framework

Learning Objectives

- Apply a systematic approach to unfamiliar ML problems.
- Justify methodological choices.
- Evaluate models critically.
- Communicate technical findings clearly.

1.11.1 Step 1: Understand the Problem

Identify:

- organizational objective;
- prediction target;
- prediction time;
- available information;
- intended decision.

Ask: What exactly must the model predict?

1.11.2 Step 2: Understand the Data

Investigate:

- observations;
- variables;
- data types;
- missing values;
- duplicates;
- distributions;
- unusual values;
- categorical variables;
- timestamps.

1.11.3 Step 3: Define Features and Target

Clearly distinguish:

X = predictor variables

y = target variable

Remove variables that should not be used, such as:

- identifiers;
- post-outcome information;
- leaked variables;
- inappropriate administrative fields.

1.11.4 Step 4: Explore the Data

Use:

- descriptive statistics;
- distributions;
- category frequencies;
- scatterplots;
- boxplots;
- correlation analysis.

Do not interpret every available variable mechanically. Focus on findings relevant to the problem.

1.11.5 Step 5: Prepare the Data

Depending on the dataset:

- remove or investigate duplicates;
- handle missing values;
- clean categories;
- encode categorical variables;
- scale numerical variables where appropriate;

- engineer useful features.

1.11.6 Step 6: Split the Data

Separate training data from unseen evaluation data. For temporal problems, consider whether chronological splitting is more appropriate than random splitting.

1.11.7 Step 7: Build Candidate Models

Select models appropriate for:

- target type;
- dataset characteristics;
- practical objective.

Do not assume that the most complicated model is the best model.

1.11.8 Step 8: Evaluate

Use appropriate metrics.

For regression:

- MAE;
- MSE;
- RMSE;
- R^2 .

1.11.9 Step 9: Compare and Critically Evaluate

Ask:

- Does the model generalize?
- Is it better than a baseline?
- Are errors acceptable?
- Are there signs of overfitting?
- Is the model stable?

- Is it sufficiently interpretable?
- Does it perform similarly across relevant groups?

1.11.10 Step 10: Consider Deployment

Before deployment, consider:

- data availability;
- privacy;
- fairness;
- reliability;
- monitoring;
- computational resources;
- human oversight;
- changes in future data.

A model is not finished simply because its Python code runs successfully.

1.11.11 Worked / Applied Example

Example: Predicting Delivery Duration

A logistics company wants to estimate the duration of deliveries.

Available information includes:

- delivery distance;
- traffic level;
- weather;
- road category;
- previous delivery duration;
- driver experience.

Problem formulation

The target is delivery duration.

Because duration is numerical, the task is a regression problem.

Data preparation

The analyst should investigate:

- missing values;
- duplicate deliveries;
- categorical road variables;
- extreme journey durations;
- variables that may contain future information.

Feature engineering

Possible features might include:

- average previous delivery duration;
- distance per expected hour;
- time-related variables.

The analyst must ensure that every engineered feature would have been available at prediction time.

Modelling

Possible models include:

- Linear Regression;
- Ridge Regression;
- Lasso Regression;
- tree-based methods;
- neural networks.

The model choice should be justified rather than assumed.

Evaluation

Possible measures include:

- MAE;
- RMSE;
- R^2 .

The analyst should determine whether the resulting errors are acceptable for the intended operational decision.

Interpretation

A model with slightly better predictive performance may not necessarily be the preferred model if the improvement is very small but the model is substantially more difficult to interpret or maintain.

1. 11.12 Exercises

- 1.** A company wants to predict the amount a customer will spend next month. Is this naturally a classification or regression problem? Explain the basis for your decision.
- 2.** Give three examples of features that could be used to predict customer behaviour.
- 3.** Explain why a missing income value should not automatically be interpreted as zero.
- 4.** Give one example of a nominal categorical variable and one example of an ordinal categorical variable.
- 5.** Explain why arbitrary numerical encoding can be problematic for nominal variables.
- 6.** What is data leakage? Give an example involving information that becomes available only after a prediction has been made.
- 7.** Explain why preprocessing parameters should normally be learned from training data.
- 8.** What is multicollinearity, and why can it create problems for regression models?
- 9.** Explain one important difference between Ridge and Lasso Regression.
- 10.** Why can accuracy be misleading when one class is much more common than another?
- 11.** Explain why correlation between two variables does not necessarily demonstrate causation.
- 12.** Give three visualizations that could be used during EDA and explain what each one is intended to investigate.

1.12 Practical Laboratory Exercise

- **Practical 1: Explore a house prediction price dataset**

Tasks:

1. Get public dataset from an online data repository, such as Kaggle dataset repository.
2. Load a CSV with Pandas.
3. Display the first five rows, last five rows, shape, data types and summary statistics.
4. Display frequency of bedrooms.
5. Plot *Area* against *Price*.
6. Identify which columns are possible features and which is the target.

Starter code:

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
import kagglehub
path = kagglehub.dataset_download("zafarali27/house-price-prediction-dataset")
print("Path to dataset files:", path)
```

```
!ls /kaggle/input/house-price-prediction-dataset
```

```
df = pd.read_csv("/kaggle/input/house-price-prediction-dataset/House Price Prediction Dataset.csv")
```

```
print(df)

print(df.head())
print(df.tail())
print(df.info())
print(df.describe())
```

```
print(df["Bedrooms"].value_counts())
```

```
df.plot.scatter(x="Area", y="Price")
plt.show()
```

Expected interpretation:

Students should explain dataset size, variable types, possible relationships and any obvious quality problems before proposing a model.

- **Practical 2: Explore a different house prediction price dataset**

Tasks:

1. Get public dataset from an online data repository, such as Kaggle dataset repository.
2. Load a CSV with Pandas.
3. Display the first five rows, last five rows, shape, data types and summary statistics.
4. Display frequency of view.
5. Plot *sqft_living* against *price*.
6. Plot histogram of *price*.
7. Plot boxplot of *price_per_sqft*
8. Create a Pipeline
6. Identify which columns are possible features and which is the target.

Starter code:

```
from google.colab import drive
drive.mount('/content/gdrive')
```

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Run the following code two times (i.e., until 'Path to dataset files:
/kaggle/input/house-price-prediction' is displayed)
```

```
import kagglehub
path = kagglehub.dataset_download("debayank2024/house-price-prediction")
print("Path to dataset files:", path)
```

```
!ls /kaggle/input/house-price-prediction
```

```
!cp /kaggle/input/house-price-prediction/modified_data.csv /content/
```

```
df = pd.read_csv("modified_data.csv")
```

```
print(df.head())
print(df.info())
```

```
print(df.describe())
```

```
print(df["view"].value_counts())
```

```
plt.scatter(df['sqft_living'], df['price'])
plt.show()
```

```
plt.hist(df['price'], bins=1000)
plt.xlabel('Price')
plt.ylabel('Count')
```

```
plt.boxplot(df['price_per_sqft'])
```

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
```

```
numeric = Pipeline([("imputer", SimpleImputer(strategy="median")), ('scaler',
StandardScaler())])
categorical= Pipeline([("Imputer", SimpleImputer(strategy="most_frequent")), ("onehot",
OneHotEncoder(handle_unknown="ignore"))])
```

```
num_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
cat_cols = df.select_dtypes(include=['object']).columns.tolist()

pp = ColumnTransformer([('numerical', numeric, num_cols), ('category', categorical,
cat_cols)])

print(pp)
```

```
pp
```

Expected interpretation:

Students should describe the dataset size and variable types, interpret the summary statistics and category frequencies, identify potential relationships and outliers from the visualizations, and note any data-quality issues that should be addressed during preprocessing before model development.

- **Practical 3: Clean and preprocess a mixed dataset.**

Tasks:

1. Load a CSV containing a house price prediction dataset.
2. Check missing values and duplicates.
3. Separate numerical and categorical columns.
4. Split into train and test sets.
5. Build a Scikit-learn ColumnTransformer using median imputation + StandardScaler for numeric columns and most-frequent imputation + OneHotEncoder for categorical columns.
6. Fit preprocessing only on training data.
7. Print the transformed feature matrix shape.

Starter code:

```
from google.colab import drive
drive.mount('/content/gdrive')
```

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Run the following code two times (i.e., until 'Path to dataset files:
/kaggle/input/house-price-prediction' is displayed)
```

```
import kagglehub
path = kagglehub.dataset_download("debayank2024/house-price-prediction")
print("Path to dataset files:", path)
```

```
!ls /kaggle/input/house-price-prediction
```

```
!cp /kaggle/input/house-price-prediction/modified_data.csv /content/
```

```
df = pd.read_csv("modified_data.csv")
```

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

```
# The following must be run once, otherwise an error will be thrown given that X will not
be having anymore the variable price
X = df.drop(columns=["price"])
```

```
y = df["price"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42) #, stratify=y)
```

```
num_cols = X.select_dtypes(include="number").columns
cat_cols = X.select_dtypes(exclude="number").columns

numeric = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
categorical = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])
preprocess = ColumnTransformer([
    ("num", numeric, num_cols),
    ("cat", categorical, cat_cols)
])
```

```
X_train_ready = preprocess.fit_transform(X_train)
X_test_ready = preprocess.transform(X_test)
```

Expected interpretation:

Students should explain the purpose of the training-test split and demonstrate that imputation, scaling, and encoding are learned from the training data and consistently applied to the unseen test data. They should explain how this prevents data leakage and provides a more reliable estimate of performance on new data.

- **Practical 4: Build a house-price regression model (continuity of Practical 3).**

Tasks:

1. Use a dataset for house prediction price.
2. Split data into training and testing sets.
3. Fit LinearRegression.
4. Predict test prices.

5. Calculate MAE, RMSE and R-squared.
6. Plot actual versus predicted values.
7. Fit Ridge regression and compare results.

Starter code:

```
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
```

```
model = LinearRegression()
```

```
model.fit(X_train_ready, y_train)
```

```
pred = model.predict(X_test_ready)
```

```
print("MAE:", mean_absolute_error(y_test, pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, pred)))
print("R2:", r2_score(y_test, pred))
```

```
ridge = Ridge(alpha=1.0)
ridge.fit(X_train_ready, y_train)
```

```
pred_rg = ridge.predict(X_test_ready)
print("MAE:", mean_absolute_error(y_test, pred_rg))
print("RMSE:", np.sqrt(mean_squared_error(y_test, pred_rg)))
print("R2:", r2_score(y_test, pred_rg))
```

```
# The following is not important for prediction, just for regression analysis
```

```
# PARAMETER INTERPRETATION & COEFFICIENT COMPARISON
```

```
# =====
```

```
# The coefficients must be interpreted with respect to the transformed feature space.
```

```
# Because the numerical variables were standardized, an OLS or Ridge coefficient represents
the change in predicted price associated with a one-standard-deviation increase in that
numerical feature, holding the other transformed features constant. For one-hot encoded
categorical variables, the coefficient represents the difference relative to the
omitted/reference category, subject to the encoding configuration.
```

```
feature_names = preprocess.get_feature_names_out()
```

```
feature_names = [
    name.replace("num__", "").replace("cat__", "")
    for name in feature_names
]
```

```
coef_comparison = pd.DataFrame({
    'Feature': feature_names,
```

```

    'OLS_Coefficients': model.coef_,
    'Ridge_Coefficients': ridge.coef_
})

print("\n=====")
print("COEFFICIENT STRUCTURAL SHRINKAGE AUDIT")
print("=====")

print(coef_comparison.to_string(index=False))

# When comparing OLS and Ridge coefficients, we can demonstrate coefficient shrinkage: This
# allows us to see directly how Ridge generally pulls coefficients toward zero, although it
# does not necessarily shrink every individual coefficient by the same amount.

coef_comparison['Absolute_OLS'] = abs(coef_comparison['OLS_Coefficients'])
coef_comparison['Absolute_Ridge'] = abs(coef_comparison['Ridge_Coefficients'])

coef_comparison['Shrinkage'] = (
    coef_comparison['Absolute_OLS']
    - coef_comparison['Absolute_Ridge']
)

print(coef_comparison.to_string(index=False))

```

Discussion:

Which model generalizes better? Are the errors acceptable for the intended decision?
Which observations have the largest residuals, and why might that happen?

- **Practical 5: End-to-End Regression Analysis**

Use a suitable numerical dataset that contains several predictor variables and a continuous target.

Tasks

1. Load the dataset using Pandas.
2. Display:
 - first five observations;
 - dataset shape;
 - data types;

- summary statistics.
3. Investigate:
 - missing values;
 - duplicate records;
 - categorical variables;
 - possible outliers.
 4. Identify the predictor variables and target variable.
 5. Produce at least three meaningful EDA visualizations.
 6. Calculate a correlation matrix for numerical variables.
 7. Identify possible highly correlated predictors.
 8. Split the dataset into training and testing sets.
 9. Prepare numerical and categorical variables appropriately.
 10. Build a Linear Regression model.
 11. Evaluate the model using:
 - MAE;
 - RMSE;
 - R^2 .
 12. Build a Ridge Regression model.
 13. Build a Lasso Regression model.
 14. Compare the models.
 15. Investigate whether changing the regularization strength affects performance.
 16. Discuss whether the best numerical result is necessarily the best practical model.

Suggested starter code

```
import pandas as pd
import numpy as np
```

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

df = pd.read_csv("your_dataset.csv")
print(df.head())
print(df.shape)
print(df.info())
print(df.describe())

X = df.drop(columns=["target"])
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
mae = mean_absolute_error(y_test, predictions)
rmse = np.sqrt(mean_squared_error(y_test, predictions))
r2 = r2_score(y_test, predictions)
print("MAE:", mae)
print("RMSE:", rmse)
print("R-squared:", r2)

```

Discussion

Students should consider:

- What does the target represent?
- Are the selected features available at prediction time?
- Were missing values handled appropriately?
- Are categorical variables encoded correctly?
- Is scaling required?
- Are some predictors highly correlated?
- Which model generalizes best?
- Are the observed errors acceptable?

- Does the model have practical limitations?
- What additional evidence would be required before deployment?

1.13 Key Principles to Remember

1. Start with the problem, not the algorithm.
2. The target determines whether a supervised problem is classification or regression.
3. Understand the data before modelling.
4. Missing does not automatically mean zero.
5. Nominal categories should not normally be assigned arbitrary numerical orders.
6. Fit preprocessing using training data and apply the learned transformations to unseen data.
7. Always consider the possibility of data leakage.
8. Correlation indicates association, not causation.
9. Highly correlated predictors can create multicollinearity problems.
10. Feature selection should consider more than correlation with the target.
11. Ridge uses L2 regularization; Lasso uses L1 regularization.
12. Regularized linear models generally benefit from appropriate feature scaling.
13. Evaluate models using metrics appropriate to the problem.
14. A model that performs well on training data may still generalize poorly.
15. Overall performance can hide poor performance for particular groups.
16. A technically accurate model may still be unsuitable for deployment.
17. Predictions support decisions; they do not automatically determine decisions.
18. Statistical association should not automatically be interpreted as causation.
19. The simplest model that performs adequately may sometimes be preferable to a more complicated model.
20. A good ML analysis explains not only what the model predicts, but also why the modelling decisions are appropriate and what limitations remain.